

ui.separator 全面详解

一、核心概述

`ui.separator` 是 NiceGUI 框架中的分隔线组件，基于 Quasar 的 `QSeparator` 组件实现，功能类似 HTML 中的 `<hr>` 标签，主要用于在卡片、菜单及其他组件容器中创建视觉分隔，提升界面布局的层次感和可读性。其核心价值是简化界面元素的分区设计，无需手动编写 CSS 分隔样式，仅需一行代码即可快速插入标准化分隔线。

二、基础用法

1. 最简示例

通过导入 `nicegui` 模块，直接调用 `ui.separator()` 即可在两个元素间插入分隔线，代码如下：

```
from nicegui import ui

ui.label('上方文本')
ui.separator() # 插入水平分隔线
ui.label('下方文本')

ui.run()
```

运行后将展示“上方文本”→分隔线→“下方文本”的垂直布局，分隔线默认占满父容器宽度，样式为浅灰色细线条。

2. 核心特性

- 轻量无依赖：无需额外配置，调用即生效；
- 自适应布局：默认继承父容器的宽度（水平方向）或高度（垂直方向，需通过 props 配置）；
- 兼容性强：可与 NiceGUI 所有组件（如 `ui.card`、`ui.menu`、`ui.row` 等）搭配使用。

三、属性 (Properties)

`ui.separator` 继承了 NiceGUI 基础元素的核心属性，支持样式定制、状态管理等扩展能力，关键属性如下：

属性名	类型	说明	新增版本
<code>classes</code>	<code>Classes[Self]</code>	为分隔线添加 HTML 类（支持 Tailwind、Quasar 类），用于批量定制样式	-
<code>client</code>	<code>Client</code>	绑定该元素所属的客户端实例（用于多客户端场景）	-
<code>html_id</code>	<code>str</code>	设定元素在 HTML DOM 中的唯一 ID，用于 JS 操作或样式定位	2.16.0
<code>is_deleted</code>	<code>bool</code>	只读属性，标识元素是否已被删除	-

属性名	类型	说明	新增版本
<code>is_ignoring_events</code>	<code>bool</code>	只读属性，标识元素是否正在忽略事件	-
<code>parent_slot</code>	<code>`Slot`</code>	<code>None`</code>	可设置属性，指定元素所属的父插槽（用于复杂组件嵌套）
<code>props</code>	<code>Props[Self]</code>	配置 Quasar 组件原生属性（如方向、颜色、厚度等）	-
<code>style</code>	<code>Style[Self]</code>	直接设置内联 CSS 样式，优先级高于 <code>classes</code>	-
<code>visible</code>	<code>BindableProperty</code>	绑定元素可见性（支持双向绑定），默认值为 <code>True</code>	-

常用属性示例

(1) 通过 `props` 定制方向和样式

利用 Quasar 原生属性配置垂直分隔线、颜色和厚度：

```
from nicegui import ui

with ui.row(): # 水平布局容器
    ui.label('左侧内容')
    ui.separator(props='vertical color="primary" thickness="2px"') # 垂直分隔线，主色调，2px 粗
    ui.label('右侧内容')

ui.run()
```

- `vertical`：将分隔线方向改为垂直（默认水平）；
- `color`：支持 Quasar 预定义颜色（如 `primary`、`secondary`、`red-500`）或十六进制颜色；
- `thickness`：设置分隔线厚度（默认 1px）。

(2) 通过 `style` 设置内联样式

直接定制分隔线的宽度、边距和颜色：

```
ui.separator(style='width: 50%; margin: 20px auto; background-color: #ff5722;')
```

- `width: 50%`：分隔线占父容器 50% 宽度；
- `margin: 20px auto`：上下边距 20px，水平居中；
- `background-color`：覆盖默认背景色（优先级高于 `props` 中的 `color`）。

(3) 通过 `html_id` 定位元素

```
ui.separator(html_id='custom-sep')  
# 可在 JS 中通过 document.getElementById('custom-sep') 操作该元素
```

四、核心方法 (Methods)

`ui.separator` 提供了丰富的方法用于动态控制元素状态、绑定事件、资源管理等，以下是高频使用的方法分类详解：

1. 可见性控制

(1) `set_visibility(visible: bool)`

直接设置分隔线是否可见：

```
sep = ui.separator()  
sep.set_visibility(False) # 隐藏分隔线
```

(2) 双向绑定可见性: `bind_visibility()`

将分隔线可见性与目标对象的属性绑定（支持双向同步）：

```
from nicegui import ui  
  
class AppState:  
    def __init__(self):  
        self.show_sep = True  
  
state = AppState()  
  
ui.checkbox('显示分隔线', value=state.show_sep, on_change=lambda e: setattr(state,  
'show_sep', e.value))  
ui.separator().bind_visibility(state, 'show_sep') # 勾选框与分隔线可见性联动  
ui.label('分隔线下方内容')  
  
ui.run()
```

- 当勾选框状态改变时，`state.show_sep` 会同步更新，分隔线的可见性随之变化；
- `forward/backward` 参数可添加值转换逻辑（如 `forward=lambda x: not x` 实现反向绑定）。

(3) 单向绑定: `bind_visibility_from()/bind_visibility_to()`

- `bind_visibility_from(target_object, target_name)`：仅从目标对象同步可见性（目标→元素）；
- `bind_visibility_to(target_object, target_name)`：仅从元素同步可见性（元素→目标）。

2. 样式与属性动态修改

(1) `default_classes()` / `default_props()` / `default_style()`

用于批量修改该类所有实例的默认样式（需在元素实例化前调用）：

```
from nicegui import ui

# 全局修改所有 ui.separator 的默认样式
ui.separator.default_classes(add='my-sep') # 添加自定义类
ui.separator.default_props(add='color="blue" thickness="3px"') # 默认蓝色、3px 粗
ui.separator.default_style(add='margin: 10px 0;') # 默认上下边距 10px

ui.label('文本1')
ui.separator() # 继承上述默认样式
ui.label('文本2')
ui.separator() # 同样继承默认样式

ui.run()
```

- `add`：新增样式 / 属性；
- `remove`：移除默认样式 / 属性（如 `remove='quasar-class'`）；
- `replace`：替换所有默认样式 / 属性（如 `replace='bg-red-500'`）；
- `toggle`：切换样式（版本 2.7.0+ 支持）。

(2) `update()`

手动触发元素在客户端的更新（修改属性后需调用以生效）：

```
sep = ui.separator()
sep.style = 'background-color: green;'
sep.update() # 同步样式修改到前端
```

3. 事件与交互

(1) `on(type: str, handler: Callable)`

为分隔线绑定事件（如点击、鼠标悬浮等）：

```
from nicegui import ui

def on_sep_click(e):
    ui.notify('分隔线被点击了！')

ui.separator().on('click', on_sep_click)
ui.label('点击上方分隔线触发通知')

ui.run()
```

- 支持的事件类型：`click`、`mousedown`、`mouseover` 等原生 DOM 事件；

- `js_handler` 参数可设置客户端 JS 处理逻辑（无需请求服务器）：

```
ui.separator(js_handler='(e) => alert("客户端触发：分隔线被点击")')
```

(2) `tooltip(text: str)`

为分隔线添加悬浮提示：

```
ui.separator().tooltip('这是一条分隔线') # 鼠标悬浮时显示提示文本
```

4. 元素管理

(1) `delete()`

删除分隔线及所有子元素（不可逆）：

```
sep = ui.separator()
sep.delete() # 从 DOM 中移除该元素
```

(2) `move(target_container: Element, target_index: int, target_slot: str)`

移动分隔线到其他容器：

```
from nicegui import ui

with ui.card() as card1:
    ui.label('卡片1')
    sep = ui.separator()

with ui.card() as card2:
    ui.label('卡片2')

# 将分隔线从 card1 移动到 card2 的末尾
sep.move(target_container=card2, target_index=-1)

ui.run()
```

(3) `ancestors()` / `descendants()`

遍历元素的祖先 / 后代元素（用于复杂组件树操作）：

```
sep = ui.separator()
# 遍历所有祖先元素（不包含自身）
for ancestor in sep.ancestors(include_self=False):
    print(ancestor)
```

5. 资源与插槽

(1) `add_resource(path: str | Path)`

为分隔线添加资源（如自定义 CSS/JS 文件）：

```
ui.separator().add_resource('./custom-styles/sep.css') # 引入外部样式文件
```

(2) `add_slot(name: str, template: str)`

添加 Vue 插槽（用于复杂组件扩展，分隔线默认无需插槽，仅在自定义组件时使用）：

```
sep = ui.separator()
# 添加名为 "custom" 的插槽，使用 Vue 模板
sep.add_slot('custom', '<span>自定义插槽内容</span>')
```

五、高级应用场景

1. 卡片内分区分隔

```
from nicegui import ui

with ui.card(title='用户信息卡片'):
    ui.label('基本信息')
    ui.separator(props='color="secondary"') # 次要颜色分隔线
    ui.label('姓名: 张三')
    ui.label('年龄: 25')

    ui.separator(props='vertical=False margin: 15px 0;') # 增加上下边距
    ui.label('账户信息')
    ui.separator(props='color="secondary"')
    ui.label('余额: 1000 元')

ui.run()
```

2. 水平布局中的垂直分隔

```
from nicegui import ui

with ui.row(style='align-items: center; gap: 20px;'):
    ui.button('按钮1')
    ui.separator(props='vertical thickness="1px" color="#e0e0e0"') # 垂直细分隔线
    ui.button('按钮2')
    ui.separator(props='vertical thickness="1px" color="#e0e0e0"')
    ui.button('按钮3')

ui.run()
```

3. 动态控制分隔线样式

结合输入组件实现实时样式调整：

```
from nicegui import ui

sep = ui.separator()

# 滑块控制分隔线厚度
ui.slider(min=1, max=10, value=1, on_change=lambda e: sep.set_prop('thickness',
f'{e.value}px'))
# 颜色选择器控制分隔线颜色
ui.color_input(value='#666666', on_change=lambda e: sep.set_prop('color', e.value))

ui.run()
```

六、注意事项

1. 方向兼容性： `vertical` 垂直分隔线需在水平布局容器（如 `ui.row`）中使用，否则可能显示异常；
2. 样式优先级： `style` 内联样式 > `classes` > `default_style` > 框架默认样式；
3. 绑定时效性： `bind_visibility` 双向绑定会实时同步，若需延迟同步可结合 `throttle` 参数（事件绑定中）；
4. 版本差异： 部分属性（如 `html_id`）和方法（如 `toggle` 类操作）需 NiceGUI 2.16.0+ 或 2.7.0+ 版本支持，使用前需确认环境版本。

总结

`ui.separator` 是 NiceGUI 中简洁高效的分隔组件，兼具基础分隔功能和灵活的扩展能力。通过 `props`、`style`、`classes` 可快速定制样式，通过绑定、事件、元素管理方法可实现动态交互，适用于各类界面布局的分区设计。其核心优势在于“零配置上手、高自由度定制”，既满足简单场景的快速使用，也支持复杂场景的深度扩展，是界面开发中提升可读性的核心组件之一。